



Tecnicatura Universitaria en Programación a Distancia

Materia: Programación 1

Trabajo Práctico Integral

Gestión de datos de países en Python: filtros, ordenamientos y estadísticas

Información personal: GRUPO 212

Alumno: Patricio Elizaincin

Alumna: Nancy Mariana Fernandez

Comisión: 11

Comisión: 19

Equipo docente:

Coordinador: Alberto Cortez

Tutores: C11 – Martina Zabala

C19 – Flor Camila Gubiotti

11 de junio de 2026

TABLA DE CONTENIDO

INTRODUCCIÓN.....	3
MARCO TEÓRICO	3
DECISIONES TÉCNICAS Y ARQUITECTURA DEL SISTEMA	5
DESARROLLO DE LAS FUNCIONALIDADES E IMPLEMENTACIÓN	6
DIFICULTADES ENCONTRADAS	9
CONCLUSIONES.....	9
ENLACES AL PROYECTO.....	10
BIBLIOGRAFÍA.....	10

INTRODUCCIÓN

El presente Trabajo Práctico Integrador tiene como objetivo desarrollar una aplicación en Python para la gestión de información de países.

El sistema permite almacenar, consultar y analizar información relacionada con distintos países utilizando estructuras de datos vistas durante la cursada de Programación I.

Para el desarrollo del proyecto se utilizaron listas, diccionarios, funciones, estructuras condicionales, estructuras repetitivas y archivos CSV, permitiendo aplicar de forma práctica los conocimientos adquiridos durante la materia.

Las funcionalidades implementadas incluyen:

- Carga de datos desde un archivo CSV.
- Alta de nuevos países.
- Actualización de información existente.
- Búsquedas por nombre.
- Filtrado por distintos criterios.
- Ordenamiento de registros.
- Obtención de estadísticas.

Este proyecto busca demostrar la aplicación integrada de los conceptos fundamentales de programación mediante una solución modular y funcional.

MARCO TEÓRICO

1) LISTAS

Las listas en Python son un tipo de dato que permite almacenar datos de cualquier tipo. Son mutables y dinámicas.

Son uno de los tipos o estructuras de datos más versátiles del lenguaje, ya que permiten almacenar un conjunto arbitrario de datos. Es decir, podemos guardar en ellas prácticamente lo que sea.

En este proyecto se utilizó una lista para almacenar todos los países cargados desde el archivo CSV.

Ejemplo conceptual:

```
países = []
```

Cada elemento de la lista representa un país.

2) DICCIONARIOS

Los diccionarios en Python son una estructura de datos que permite almacenar su contenido en forma de llave y valor.

Es una colección de elementos, donde cada uno tiene una llave “key” y un valor “value”. Los diccionarios se pueden crear con llaves {} separando con una coma cada par “key: value”.

En este proyecto cada país fue representado mediante un diccionario que contiene:

- Nombre
- Población
- Superficie
- Continente

Ejemplo:

```
{  
  "nombre": "Argentina",  
  "población": 45376763,  
  "superficie": 2780400,  
  "continente": "América"  
}
```

Esta estructura facilita el acceso y actualización de la información.

3) FUNCIONES

Una función es un bloque de código reutilizable que realiza una tarea específica. En lugar de escribir el mismo código varias veces, podemos definir una función y llamarla cuando sea necesario.

Es decir, permiten dividir un programa en bloques independientes de código que realizan tareas específicas.

Su utilización mejora:

- La reutilización del código.
- La organización del programa.
- La legibilidad.
- El mantenimiento.

Algunas funciones desarrolladas en este proyecto fueron:

```
cargar_paises()
guardar_paises()
agregar_pais()
actualizar_pais()
buscar_pais()
filtrar_paises()
ordenar_paises()
estadísticas()
```

4) ESTRUCTURAS CONDICIONALES

Una estructura condicional permite ejecutar diferentes bloques de código según si una condición se cumple o no. Esto es fundamental para la toma de decisiones en un programa.

Fueron utilizadas para:

- Validar datos ingresados.
- Controlar opciones del menú.
- Verificar búsquedas.
- Validar filtros.

Se implementaron mediante:

```
if
elif
else
```

5) ESTRUCTURAS REPETITIVAS

Las estructuras repetitivas permiten ejecutar instrucciones múltiples veces mediante iteraciones.

En programación, la iteración es el proceso de repetir una serie de instrucciones múltiples veces hasta que se cumpla una condición específica. Esta característica es fundamental porque permite automatizar tareas repetitivas, optimizar el código y reducir la cantidad de líneas necesarias para resolver problemas.

En el proyecto se utilizaron:

- **While** → Para mantener activo el menú principal hasta que el usuario decida finalizar la aplicación.
- **For** → Para:
 - Recorrer la lista de países.
 - Ordenar elementos.
 - Buscar registros.
 - Generar estadísticas.
 - Filtrar información.

6) ORDENAMIENTOS

Los algoritmos de ordenamiento permiten reorganizar los datos según determinados criterios.

En este proyecto se implementó un algoritmo basado en comparaciones e intercambios de posiciones.

Los criterios disponibles son:

- Nombre
- Población
- Superficie

Cada criterio puede ordenarse en forma:

- Ascendente
- Descendente

7) ESTADÍSTICAS BÁSICAS

Las estadísticas permiten obtener información resumida del conjunto de datos.

El sistema calcula:

- País con mayor población.
- País con menor población.
- Promedio de población.
- Promedio de superficie.
- Cantidad de países por continente.

Estas operaciones permiten analizar la información almacenada de forma sencilla.

8) ARCHIVOS CSV

CSV (Comma Separated Values) es un formato de almacenamiento de datos ampliamente utilizado.

En este proyecto se empleó para almacenar permanentemente la información de los países.

Las ventajas del uso de CSV son:

- Facilidad de lectura.
- Compatibilidad con múltiples aplicaciones.
- Persistencia de datos.

La lectura y escritura del archivo se realizó mediante el módulo:

```
import csv
```

DECISIONES TÉCNICAS Y ARQUITECTURA DEL SISTEMA

1) DECISIONES TÉCNICAS

Para el desarrollo del sistema se tomaron diversas decisiones técnicas con el objetivo de mejorar la organización, legibilidad y mantenimiento del código.

Se utilizó una lista para almacenar todos los países cargados en memoria, permitiendo recorrer fácilmente los registros y aplicar operaciones de búsqueda, filtrado y ordenamiento.

Cada país fue representado mediante un diccionario, ya que esta estructura permite acceder a la información mediante claves descriptivas, facilitando la manipulación de los datos.

Para la persistencia de la información se eligió el formato CSV debido a su simplicidad y compatibilidad con Python mediante el módulo csv.

Asimismo, el programa fue dividido en funciones independientes para cumplir con el principio de modularidad, asignando una responsabilidad específica a cada función.

Finalmente, se implementó manejo de excepciones mediante bloques try-except para controlar posibles errores durante la lectura de archivos y el ingreso de datos por parte del usuario.

2) ARQUITECTURA DEL SISTEMA

La aplicación se encuentra estructurada en módulos funcionales implementados mediante funciones independientes.

Al iniciar el programa se ejecuta la función `cargar_paises()`, encargada de leer el archivo CSV y cargar la información en memoria utilizando una lista de diccionarios.

Posteriormente se presenta un menú principal que permite acceder a las distintas funcionalidades del sistema:

- Agregar países.
- Actualizar información existente.
- Realizar búsquedas.
- Aplicar filtros.
- Ordenar registros.
- Obtener estadísticas.

Todas estas operaciones trabajan sobre la misma estructura de datos almacenada en memoria.

Cuando el usuario selecciona la opción de salida, el sistema ejecuta la función `guardar_paises()`, que persiste los cambios realizados escribiendo nuevamente la información en el archivo CSV.

Esta arquitectura permite separar responsabilidades, mejorar la legibilidad del código y facilitar futuras modificaciones o ampliaciones del sistema.

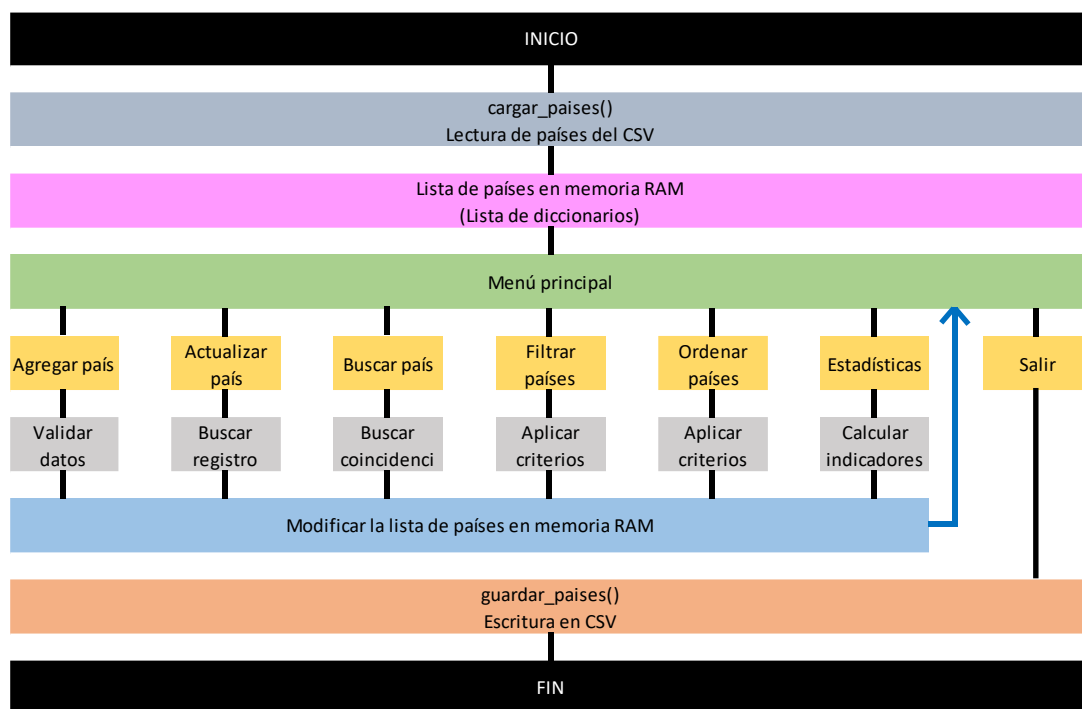


Imagen 1. Ejemplo

DESARROLLO DE LAS FUNCIONALIDADES E IMPLEMENTACIÓN

1) CARGA DE DATOS

Al iniciar el programa se realiza la lectura del archivo CSV utilizando DictReader.

Cada fila es convertida en un diccionario y almacenada dentro de una lista.

2) ALTA DE PAÍSES

Permite incorporar nuevos países verificando que no existan campos vacíos y validando los tipos de datos numéricos.

A continuación se visualiza el programa siendo ejecutado desde la terminal:

```

----- MENÚ PRINCIPAL -----
1 - Agregar país
2 - Actualizar país
3 - Buscar país
4 - Filtrar países
5 - Ordenar países
6 - Estadísticas
7 - Salir y Guardar
Seleccione una opción: 1
Nombre: Mexico
Población: 134000000
Superficie: 1964000
Continente: America
País agregado correctamente

```

Imagen 2. Alta de un nuevo país.

3) ACTUALIZACIÓN

Permite modificar población y superficie de un país existente.

```

----- MENÚ PRINCIPAL -----
1 - Agregar país
2 - Actualizar país
3 - Buscar país
4 - Filtrar países
5 - Ordenar países
6 - Estadísticas
7 - Salir y Guardar
Seleccione una opción: 2
Ingrese el país a actualizar: Argentina
Nueva población: 46000000
Nueva superficie: 3762000
País actualizado correctamente

```

Imagen 3. Actualización de población y superficie.

4) BÚSQUEDA

Permite localizar países mediante coincidencia parcial o exacta.

```

----- MENÚ PRINCIPAL -----
1 - Agregar país
2 - Actualizar país
3 - Buscar país
4 - Filtrar países
5 - Ordenar países
6 - Estadísticas
7 - Salir y Guardar
Seleccione una opción: 3
Nombre a buscar: ar
-----
Nombre: Argentina
Población: 46000000
Superficie: 3762000
Continente: America
-----
Nombre: Argelia
Población: 48000000
Superficie: 2381000
Continente: Africa

```

Imagen 4. Búsqueda parcial por nombre.

5) FILTRADO

Permite filtrar por:

- Continente
- Población
- Superficie

```

----- MENÚ PRINCIPAL -----
1 - Agregar país
2 - Actualizar país
3 - Buscar país
4 - Filtrar países
5 - Ordenar países
6 - Estadísticas
7 - Salir y Guardar
Seleccione una opción: 4

--- FILTRAR POR ---
1 - Continente
2 - Población
3 - Superficie
Opción: 1
Continente: America
Mexico - Pob: 134000000 - Sup: 1964000
Argentina - Pob: 46000000 - Sup: 3762000
  
```

Imagen 5. Filtrado por continente.

6) ORDENAMIENTO

Permite reorganizar la información según distintos criterios.

```

----- MENÚ PRINCIPAL -----
1 - Agregar país
2 - Actualizar país
3 - Buscar país
4 - Filtrar países
5 - Ordenar países
6 - Estadísticas
7 - Salir y Guardar
Seleccione una opción: 5

--- CRITERIO DE ORDENAMIENTO ---
1 - Nombre
2 - Población
3 - Superficie
Opción: 3

--- DIRECCIÓN ---
1 - Ascendente (Menor a Mayor / A-Z)
2 - Descendente (Mayor a Menor / Z-A)
Opción: 2

Lista ordenada:
Nombre: Argentina, Pob: 46000000, Sup: 3762000
Nombre: Argelia, Pob: 48000000, Sup: 2381000
Nombre: Mexico, Pob: 134000000, Sup: 1964000
  
```

Imagen 6. Ordenamiento por superficie descendente.

7) ESTADÍSTICAS

Calcula indicadores generales del conjunto de datos.

```
----- MENÚ PRINCIPAL -----  
1 - Agregar país  
2 - Actualizar país  
3 - Buscar país  
4 - Filtrar países  
5 - Ordenar países  
6 - Estadísticas  
7 - Salir y Guardar  
Seleccione una opción: 6  
  
--- ESTADÍSTICAS ---  
País con mayor población: Mexico (134000000)  
País con menor población: Argentina (46000000)  
Promedio de población: 76000000.00  
Promedio de superficie: 2702333.33  
  
Cantidad de países por continente:  
America: 2  
Africa: 1
```

Imagen 7. Estadísticas generadas por el sistema.

DIFICULTADES ENCONTRADAS

Durante el desarrollo del proyecto surgieron diversas dificultades relacionadas principalmente con la lectura de archivos CSV, la validación de datos ingresados por el usuario y la implementación de los algoritmos de ordenamiento.

Para resolver estos inconvenientes se incorporaron estructuras de manejo de errores mediante bloques try-except, permitiendo evitar interrupciones inesperadas del programa.

También fue necesario organizar adecuadamente las funciones para mantener un código modular y fácil de mantener.

CONCLUSIONES

La realización de este proyecto permitió integrar los principales conceptos estudiados durante la cursada de Programación I.

Se logró aplicar estructuras de datos como listas y diccionarios, implementar funciones para modularizar el código y utilizar estructuras condicionales y repetitivas para controlar el flujo del programa.

Además, se incorporó el uso de archivos CSV para la persistencia de datos y se desarrollaron funcionalidades de búsqueda, filtrado, ordenamiento y análisis estadístico.

El proyecto permitió comprender la importancia de la organización del código, la validación de datos y el manejo de errores para desarrollar aplicaciones más robustas y mantenibles.

ENLACES AL PROYECTO



[Repositorio GitHub](#)



[Vídeo demostración](#)

BIBLIOGRAFÍA

- Tecnicatura Universitaria en Programación. (s.f.). *Unidad 2: Estructuras condicionales*. Programación 1.
- Tecnicatura Universitaria en Programación. (s.f.). *Unidad 3: Estructuras repetitivas*. Programación 1.
- Tecnicatura Universitaria en Programación. (s.f.). *Unidad 6: Funciones*. Programación 1.
- Tecnicatura Universitaria en Programación. (s.f.). *Unidad 7: Estructuras de datos complejos*. Programación 1.
- Tecnicatura Universitaria en Programación. (s.f.). *Unidad 8: Manejo de errores*. Programación 1.
- Tecnicatura Universitaria en Programación. (s.f.). *Unidad 9: Manejo de archivos*. Programación 1.
- Revuelta, Á. (s.f.). *El Libro de Python*. <https://ellibrodepython.com/>